API Contracts & OpenAPI

contract-first, typed clients and Pact

Watch the 4-min video

In one line: an API contract (usually an OpenAPI file) says exactly what the API accepts and returns; generate docs, mocks and typed clients from it, diff it in CI, and use consumer-driven contract tests so a breaking change fails a build instead of a release.

1 The bug no test caught

The mobile team builds against your orders API. A backend engineer renames `total` to `total_amount`. All tests pass and the release goes out. That night the app crashes for every user. Nobody broke a test, because **nobody tested the contract**, the promise between the API and the code that calls it.

An **API contract** describes exactly what the API accepts and returns: **paths, methods, parameters, request and response schemas, status codes and authentication**. The standard format is **OpenAPI**, a YAML or JSON document (it was called Swagger before version 3). One file becomes the single source of truth for docs, clients, mocks and tests.

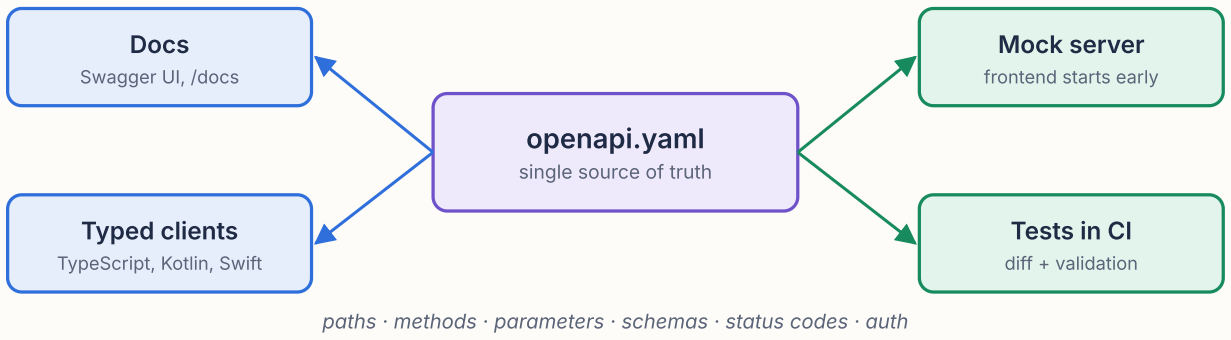


Figure 1. One reviewed file feeds everything else, so docs, clients, mocks and tests can't drift apart.

```
openapi: 3.1.0
info: {title: Orders API, version: 1.4.0}
paths:
  /orders/{id}:
    get:
      parameters: [{name: id, in: path, required: true, schema: {type: string}}]
      responses:
        '200': {content: {application/json: {schema: {$ref: '#/components/schemas/Order'}}}}
        '404': {description: Order not found}
components:
  schemas:
    Order: {type: object, required: [id, total],
            properties: {id: {type: string}, total: {type: number}}}
```

2 Code-first or contract-first

Code-first: write the code, then generate the spec. **FastAPI** does this automatically: your Pydantic models and route signatures become an OpenAPI document served at `/openapi.json`, with interactive docs at `/docs`. It is quick and always in sync with the code.

Contract-first flips it: design the spec **before** writing any code. Teams review the API shape in a pull request, frontend and backend agree, and both start in parallel, the frontend against a **mock server generated from the spec**. It is slower to start, but it catches design mistakes while they are cheap, and it suits **public and cross-team APIs**.

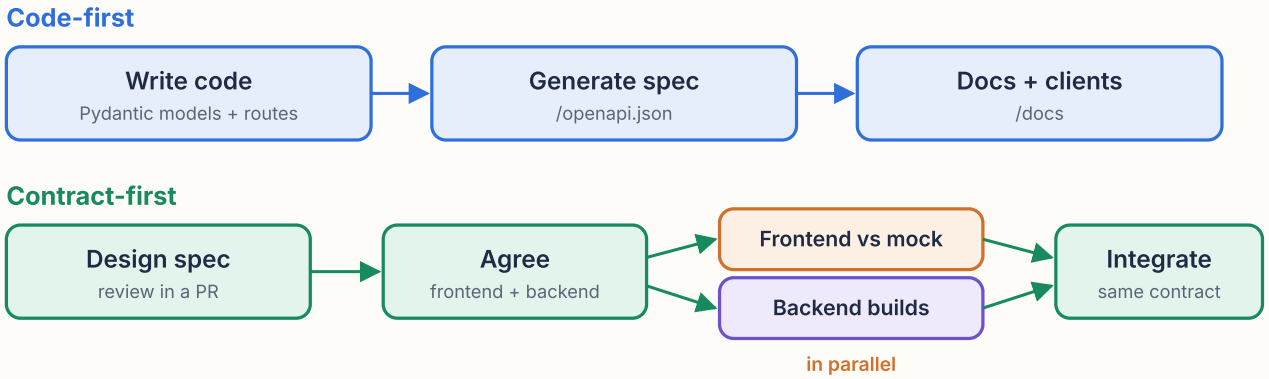


Figure 2. Code-first: the spec is an output of the code. Contract-first: the spec is the input, and both teams build against it at the same time.

	Code-first	Contract-first
Source of truth	The code	The spec file
Speed to start	Fast	Slower: design and review first
Parallel work	Frontend waits for a working backend	Frontend starts on a mock on day one
Best for	Internal APIs, one team owns both sides	Public, partner and cross-team APIs

3 Generate typed clients

Either way, **generate clients instead of hand-writing them**. From the FastAPI spec, a generator like **openapi-typescript** or **OpenAPI Generator** produces a typed TypeScript client. Now if the backend renames a field, the **frontend build fails at compile time** instead of crashing in production.

```
npx openapi-typescript https://api.example.com/openapi.json -o src/api/schema.d.ts

// after the rename, the old code no longer compiles:
const t: number = order.total;
//   error TS2339: Property 'total' does not exist on type 'Order'.
```

4 Catch breaking changes in CI

Diff the spec on every pull request. A tool like **oasdiff** compares the new OpenAPI file with the one running in production and **fails the build on breaking changes**: a removed field, a new required parameter, a changed type. **Additive changes pass**.

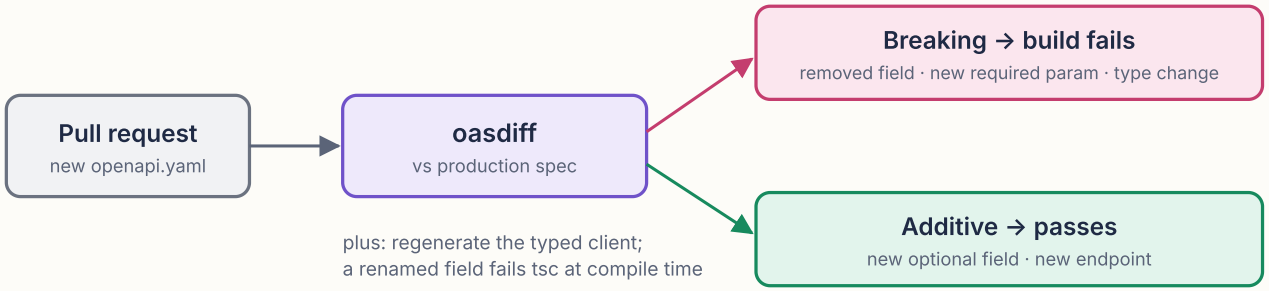


Figure 3. Spec diffing is cheap and runs in seconds. It catches the total → total_amount rename before merge, not at midnight.

5 Consumer-driven contract tests (Pact)

Between microservices, add **consumer-driven contract tests**. With **Pact**, each consumer writes tests that record what it actually uses from a provider: *this request, and these fields in the response*. Those contracts are published to a **broker**, and the provider's pipeline **verifies it still satisfies every consumer** before it deploys. A `can-i-deploy` check answers that automatically.

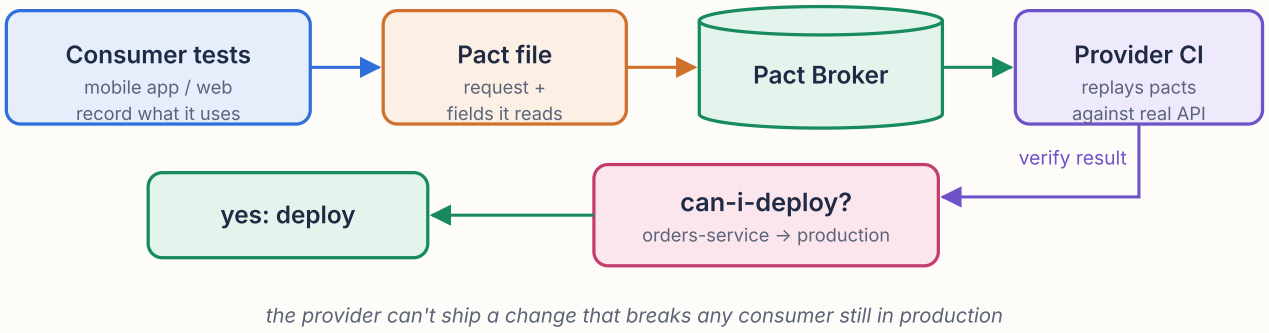


Figure 4. Each consumer states its real needs. The provider is free to change anything no consumer uses, and blocked only from changes that would break one.

WORKED EXAMPLE

The orders service has **3 consumers**: the mobile app, the web app and the billing service. The mobile pact says it reads `id`, `status` and `total`. A pull request renames `total` to `total_amount`.

- **oasdiff**: "property total removed from response" → build fails in about 10 seconds.
- **Pact verification**: the mobile pact fails; `can-i-deploy` says no.
- **Fix**: add `total_amount` and keep `total` (an additive change). Both checks pass. Remove `total` later, once no consumer reads it (next lesson).

COMMON MISTAKES

- Treating the spec as documentation written after the fact, so it drifts from the real API.
- Hand-writing client models that silently fall out of date.
- Testing only the provider's own behavior: unit tests pass while consumers break.
- Diffing against the main branch instead of what is **actually in production**.

6 Key terms

API contract: what an API accepts and returns

Contract-first: spec designed and agreed before code

oasdiff: diffs two OpenAPI files, flags breaking changes

OpenAPI: standard YAML/JSON format for HTTP API contracts

Mock server: fake API generated from the spec

Pact: consumer-driven contract testing tool

Code-first: spec generated from code

Typed client: generated code that fails to compile on mismatch

can-i-deploy: broker check: safe to release?

7 Interview questions

Q1. What is contract-first design? BASIC

Designing and agreeing on the API spec, usually OpenAPI, before writing the implementation, then generating mocks, stubs, clients and docs from it, so teams work in parallel against one source of truth.

Q2. What does an OpenAPI document contain? BASIC

Paths and methods, parameters, request and response schemas, status codes (200, 404 ...), and security schemes such as a bearer token. It is YAML or JSON and machine-readable.

Q3. Code-first or contract-first: when would you choose each? BASIC

Code-first (e.g. FastAPI generating /openapi.json) for internal APIs where one team owns both sides and speed matters. Contract-first for public, partner and cross-team APIs, where agreeing on the shape early is cheaper than changing it later.

Q4. How do you catch breaking changes before release? SCENARIO

Diff the OpenAPI spec against production in CI (oasdiff fails on removed fields, new required params, type changes); generate typed clients so mismatches fail at compile time; run consumer-driven contract tests like Pact, with can-i-deploy gating the provider's release.

Q5. A frontend team waits 3 weeks for the backend before they can start. How do you fix that? SCENARIO

Go contract-first: agree the OpenAPI spec in a PR in a day or two, generate a mock server from it, and let the frontend build against the mock while the backend implements the same spec.

Q6. You have 12 microservices calling each other. Why add Pact if you already have end-to-end tests? SCENARIO

End-to-end tests need all 12 services deployed together, are slow and flaky, and rarely cover every field. Pact tests each consumer–provider pair in isolation in seconds and tells you exactly which consumer a change breaks.

Q7. Your spec says total is required, but the server sometimes omits it. Tests pass. Why, and how do you catch it? TRICKY

Nothing checks responses against the spec. Add response validation in tests (or in staging), and contract tests that assert the fields consumers read are present. A spec that isn't enforced is just documentation.

Q8. Is adding a new required field to a request body breaking? To a response? TRICKY

Request: yes, old clients don't send it and now get 400. Response: usually safe for tolerant readers, since old clients ignore it; oasdiff treats new optional request fields and new response fields as non-breaking.

Next: SD-C-023 · API Backward Compatibility backendroadmap.com